SYSTEMPULSE

A Local-First System Intelligence and Behavioral Analysis Architecture for Evidence-Oriented System Diagnosis

Technical Research Paper

AVYAKTA NX SYSTEMS

Author: Nachiket Narkhede

Abstract

Modern operating systems expose extensive telemetry, yet raw measurements rarely provide sufficient context to determine why a machine is slow, unstable, or behaving differently from its usual operating pattern. SystemPulse investigates the engineering gap between telemetry collection and human interpretation.

The implemented platform is local-first. It collects CPU, memory, disk, network, Wi-Fi, process, device, and connection observations; maintains bounded recent histories; identifies selected deviations; and converts observations into evidence-oriented assessments and recommendations.

The current implementation is not presented as a full machine-learning anomaly-detection system. Its behavioral layer combines rolling observation windows, deterministic thresholds, deviation rules, intent classification, fuzzy matching, and evidence-oriented reasoning. Research Mode provides longitudinal telemetry for future statistical and machine-learning experiments.

Agent L is a local reasoning interface based on system state and explicit analysis logic. No unsupported benchmark, user-study, security-detection rate, or predictive-performance claim is made.

Keywords : system intelligence; behavioral analysis; telemetry; anomaly detection; local-first computing; evidence reasoning; diagnostics; edge intelligence.

1. Introduction

System monitoring traditionally focuses on visibility: expose resource measurements so an operator can observe what a machine is doing. Visibility is necessary, but a stream of numbers does not automatically answer why the system is behaving that way.

SystemPulse was structured around a different question: how can observable system state be transformed into useful contextual understanding without pretending that an observation is already a diagnosis?

```
Symptom → Observation → Evidence → Assessment → Recommendation
```

A symptom such as 'the computer is slow' is underspecified. CPU utilization, memory pressure, disk activity, process behavior, network state, Wi-Fi state, and device information can provide additional evidence.

The architecture is intentionally broader than security monitoring. High CPU can be legitimate. Unusual disk activity can result from updates or file processing. A listening endpoint can belong to a legitimate service.

The project's research position is therefore observation-first: observe, contextualize, interpret, and preserve uncertainty.

2. Problem Formulation

Let the observable system state at time t be represented as a multidimensional vector.

```
X(t) = {CPU(t), RAM(t), Disk(t), Network(t), WiFi(t), Processes(t), Devices(t),
Connections(t)}
```

A fixed threshold evaluates whether a signal crosses an absolute boundary. That is useful for immediate classification, but it does not establish whether the current state is unusual for the machine itself.

2.1 Relative behavior

```
B(t) = mean[x(t−1), x(t−2), …, x(t−n)]
```

```
D(t) = x(t) − B(t)
```

SystemPulse implements this concept for CPU using a short rolling window. It should therefore be described as short-term behavioral comparison, not as a solved long-term adaptive model.

2.2 Core questions

1. Can recent system behavior provide more context than fixed thresholds alone?

2. Can multiple observations become meaningful evidence when correlated?

3. How can an unusual observation remain distinct from a security conclusion?

4. How should baselines adapt when legitimate workloads change?

3. Research Objectives and Principles

1. Continuously observe multiple dimensions of local machine state.
2. Maintain bounded short-term histories.
3. Generate interpretable deviation events.
4. Provide evidence-oriented explanations.
5. Enable longitudinal research.
6. Keep core telemetry local.
7. Provide a foundation for multivariate and ML research.
8. Separate measured, derived, inferred, and recommended information.

3.1 Design principles

• Observation is not diagnosis.

• Anomaly is not automatically vulnerability.

• An unusual process is not automatically malware.

• A listening port is not automatically an attack.

• Unavailable data should remain unavailable.

• Complexity should be justified by measurable research benefit.

• Interpretations should remain traceable to available evidence.

These principles keep the present implementation distinct from future research proposals.

4. System Architecture

The architecture progresses from operating-system state toward human-readable system intelligence.

```
SYSTEM ↓ TELEMETRY ↓ BASELINE ↓ BEHAVIOR ↓ DEVIATION ↓ EVIDENCE ↓ RISK / CONTEXT ↓ EXPLANATION
↓ SYSTEM INTELLIGENCE
```

Figure 1. Conceptual SystemPulse intelligence pipeline.

Layer	Purpose
Machine state	Operating-system and hardware-exposed state.
Telemetry	CPU, RAM, disk, network, Wi-Fi, process and device observations.
Baseline	Recent reference behavior where implemented.
Behavior	Comparison and state classification.
Deviation	Explicit event generation.
Evidence	Relevant observations assembled for a question.
Reasoning	Contextual assessment and recommendation.
Research Mode	Longitudinal collection and export.
Interface	Human-facing visualization and interaction.

5. Implementation Architecture

The verified project contains a Python local agent, server, browser application, PWA assets, configuration, metadata, and documentation.

Component	Verified role
agent/analyzer.py	Bounded histories, CPU deviation logic, intent and reasoning logic.
agent/collectors.py	psutil telemetry, Windows Wi-Fi/device collection, processes and connections.
server.py	Telemetry worker, periodic collectors, API endpoints, research and reporting.
web/index.html	Browser interface structure.
web/app.js	Client-side behavior and visualization.
web/style.css	Dashboard presentation.
web/manifest.webmanifest	PWA metadata.
web/sw.js	Service-worker layer.
requirements.txt	Python dependencies.
README.md	Local setup and architecture.

The documented local workflow installs requirements, starts the server, and opens the browser interface through a loopback address.

6. Telemetry Acquisition

The collector uses psutil and operating-system interfaces. The verified sample includes timestamp, platform, CPU, RAM, available RAM, disk utilization, disk read/write rates, network download/upload rates, and Wi-Fi information where supported.

Component	Approximate cadence
Primary telemetry	0.5 seconds
Top processes	2 seconds
Windows devices	5 seconds
Network connections	3 seconds

Different cadences reduce unnecessary work while preserving high-frequency resource observation.

Windows Wi-Fi and device collectors depend on operating-system facilities. Their availability therefore depends on platform support, drivers, permissions, and exposed system state.

A collector reports what the operating system exposes. Absence of a value is not itself a negative observation.

7. Process, Device and Network Intelligence

7.1 Process intelligence

SystemPulse collects active-process information and uses it as evidence when interpreting resource behavior. A process can be associated with CPU or memory consumption.

A resource-heavy process is not automatically treated as malicious. Resource consumption is a measurable property; intent requires additional evidence.

7.2 Network intelligence

The connection collector exposes active internet-family connections, including endpoint and status information and process association where available.

This enables contextual questions about active endpoints without automatically converting an endpoint into a security finding.

7.3 Device intelligence

Windows device enumeration adds hardware-adjacent state. It supports investigation of device presence and operating-system-reported device information.

7.4 Cross-domain value

CPU plus process, disk plus process, and network plus process are examples of future cross-signal reasoning. Full multivariate correlation is a research direction rather than a completed capability.

8. Behavioral Baseline Implementation

The Engine stores recent CPU and RAM observations in bounded deques. The CPU history retains up to approximately 120 observations.

8.1 Short rolling CPU baseline

When enough observations exist, the implementation calculates the mean of the preceding nine CPU samples, excluding the current sample.

$$B_CPU(t) = (CPU(t-1)+CPU(t-2)+\ldots+CPU(t-9))/9$$

$$D_CPU(t) = CPU(t) - B_CPU(t)$$

8.2 Deviation rule

A CPU deviation event is generated when current CPU is at least 35 percentage points above the recent baseline. The event records its timestamp, type, baseline and current CPU value.

8.3 Interpretation

This is a deterministic, interpretable behavioral rule. It is not an ML anomaly model and not a long-term adaptive baseline.

This distinction is essential for accurately describing the verified implementation.

9. Absolute Thresholds and Behavioral Context

Signal	Current rule
CPU very high	>= 90%
CPU elevated	>= 70%
RAM very high	>= 92%
RAM elevated	>= 75%
Disk I/O burst	>= 60 MB/s
CPU deviation	Current >= baseline + 35 percentage points

```
Absolute threshold = How high is it?

Behavioral baseline = How different is it from recent behavior?
```

A machine normally operating near 10% CPU can undergo a major relative change at 60%, even without crossing a very-high threshold. Conversely, a sustained high value can be legitimate for a known workload.

The two mechanisms are therefore complementary. Threshold values are engineering rules, not experimentally validated universal limits.

Formal evaluation must measure false positives, false negatives, sensitivity to sampling interval, and workload dependence.

10. Baseline Drift and Non-Stationary Behavior

Computer behavior changes because of updates, new software, development workloads, background services, media processing, user habits, and hardware conditions.

```
Slow adaptation → persistent false alerts

Fast adaptation → possible anomaly absorption
```

The current implementation provides short-term rolling comparison but does not claim a complete long-term drift solution. Research Mode provides historical observations for future investigation.

10.1 Candidate future methods

• Exponentially weighted moving averages

• Robust moving statistics

• Change-point detection

• Workload-conditioned baselines

• Time-aware baselines

• Confidence-aware adaptation

• Explicit regime-change detection

Any adaptive method must be evaluated against independent ground truth so that the system does not simply learn to accept every persistent condition it observes.

11. Evidence-Oriented Reasoning

```
Observation → Evidence → Assessment → Recommendation
```

11.1 Observation

A directly collected value such as CPU utilization, memory utilization, process identity, network endpoint, or Wi-Fi state.

11.2 Evidence

The subset of available observations relevant to a particular question.

11.3 Assessment

A contextual statement supported by available evidence, such as elevated resource load or a recent CPU deviation.

11.4 Recommendation

A suggested investigative direction without claiming that an unobserved root cause has been proven.

11.5 Security boundary

High CPU does not automatically prove hardware failure. An unusual process does not automatically prove malware. A listening port does not automatically prove compromise.

11.6 Evidence integrity

A conclusion should remain traceable to observable state. This improves both human trust and future experimental evaluation.

12. Agent L and Intent Classification

Agent L is the local reasoning interface. The verified implementation is a rule-oriented reasoning layer, not a claim of a general-purpose LLM.

Intent	Typical evidence
CPU	CPU state, recent behavior, process activity.
Memory	RAM utilization and available memory.
Disk	Disk utilization and I/O.
Wi-Fi	Connection state and Wi-Fi information.
Network	Connections and endpoint/process context.
Device	Windows device information.
Security	Evidence-oriented system observations.
Performance	Combined resource evidence.
Action	Investigative recommendations.
Explanation	Contextual interpretation.

Intent resolution uses direct, partial and fuzzy matching. The same telemetry can have different relevance depending on the user's question.

13. Research Mode and Longitudinal Observation

Research Mode changes SystemPulse from a purely live interface into a source of structured longitudinal observations.

• Timestamped CPU and RAM observations

• Disk activity

• Network activity

• Wi-Fi state where available

• Research-session state

• CSV-exportable data

13.1 Why time matters

Behavioral intelligence depends on trajectories, not isolated points. Longitudinal data allows investigation of persistence, recovery, repeated patterns, workload transitions, and drift.

13.2 Research questions

1. How stable is a machine baseline under repeated workloads?

2. How quickly do deviations disappear after workload termination?

3. Which signals move together?

4. Which deviations are transient versus persistent?

5. Can legitimate workload transitions be identified?

6. Does adaptive modeling improve precision without reducing recall?

Research Mode is an observation source, not automatically a labeled training dataset. Ground truth and experiment metadata must be established independently.

14. Local-First and Edge Intelligence

```
Machine → Local Telemetry → Local Analysis → Local Reasoning → Browser
```

The local-first design keeps core machine telemetry and reasoning on the host. This supports privacy, low-latency interaction, offline operation, and experimental control.

14.1 Privacy

System telemetry can expose process names, applications, network endpoints, devices, and behavioral patterns. Keeping the primary pipeline local reduces unnecessary transmission.

14.2 Latency

Local processing avoids dependence on a remote service for every observation.

14.3 Edge intelligence

The architecture provides a path toward future edge systems where compact models or deterministic reasoning operate close to sensors and constrained platforms.

14.4 Security caveat

Local execution is not equivalent to complete security. External exposure would require authentication, authorization, encrypted transport and access controls.

15. Interface and Operational Workflow

15.1 Workflow

1. Start local service and workers.
2. Sample telemetry.
3. Ingest observations into bounded histories.
4. Evaluate CPU deviation.
5. Refresh process, device and connection snapshots.
6. Expose structured state through APIs.
7. Render current telemetry and intelligence.
8. Interpret questions through Agent L.
9. Record observations through Research Mode when enabled.
10. Export structured reports.

15.2 Interface functions

• Live CPU, RAM, disk and network views

• Wi-Fi information

• Live intelligence

• Agent L

• Event timeline

• Device intelligence

• Research Mode

• JSON reporting

• PWA shell

15.3 Human-in-the-loop orientation

The interface supports investigation rather than automatic high-impact remediation. The operator remains responsible for interpreting system behavior in its broader environment.

16. Evaluation Methodology

The current implementation should be evaluated as a research prototype. Controlled workloads and independent ground truth are required before performance claims can be made.

Class	Example
Idle	Minimal foreground activity.
CPU	Controlled computation or compilation.
Memory	Controlled memory pressure.
Disk	Sustained read/write workload.
Network	Controlled data transfer.
Mixed	Simultaneous resource activity.
Recovery	Return to idle.
Persistent change	Long-running new workload or software state.

16.1 Ground truth

Record event start, event end, affected subsystem, expected behavior, and whether the condition is intentionally normal or abnormal.

16.2 Avoiding circular validation

SystemPulse must not be used as its own ground truth. Otherwise evaluation becomes circular.

16.3 Repetition

Experiments should be repeated across sessions and, where possible, across different configurations.

17. Evaluation Metrics and Comparative Design

17.1 Metrics

• Detection rate

• False-positive rate

• Precision

• Recall

• F1 score

• Detection latency

• Explanation fidelity

• Evidence completeness

• Recovery characterization

17.2 Comparison

Approach	Mechanism	Question
Fixed rules	Absolute thresholds	How effective are deterministic boundaries?
Rolling baseline	Recent statistical context	Does relative behavior reduce context-poor alerts?
Multivariate statistics	Joint signal relationships	Does correlation improve detection?
Machine learning	Learned representation	Does added complexity provide measurable benefit?

17.3 Reproducibility

Preserve operating-system version, hardware configuration, sampling interval, workload definition, parameters, duration, raw telemetry, event labels and evaluation scripts.

18. Limitations and Threats to Validity

18.1 Current limitations

• CPU baseline is short-term.

• Thresholds are heuristic engineering rules.

• Cross-signal correlation is limited.

• Agent L is not a general-purpose LLM.

• Research Mode is not a complete research database.

• Hardware visibility depends on OS and drivers.

• No population-level performance study is established.

• No guarantee of malware or vulnerability detection.

• No guarantee of hardware-failure prediction.

• Not a replacement for dedicated security or hardware tools.

18.2 Threats to validity

A single machine can exhibit highly specific behavior. Workloads differ across development, gaming, rendering, compilation, media processing, virtualization and background maintenance.

Threshold performance is sensitive to sampling frequency and duration. Future ML experiments can also be affected by label quality, leakage, imbalance, changing workloads and insufficient rare-event coverage.

18.3 Scientific boundary

The correct interpretation is foundational: SystemPulse establishes instrumentation and reasoning infrastructure for deeper experiments rather than claiming final diagnostic performance.

19. Future Research Directions

19.1 Adaptive baselines

Future work can investigate robust moving statistics, exponentially weighted models, change-point detection, workload-conditioned profiles, temporal patterns and confidence-aware adaptation.

19.2 Multivariate behavior

```
Telemetry ↓ Feature Representation ↓ Baseline Model ↓ Multivariate Deviation ↓ Cross-Signal Correlation ↓ Event Classification ↓ Evidence Graph ↓ Explanation
```

Figure 2. Proposed future multivariate research architecture.

19.3 Predictive analysis

Longitudinal trajectories could be studied for early indicators of performance or reliability problems. Prediction requires independent labels, sufficient time-series data, repeated experiments and rigorous validation.

19.4 Hardware-aware intelligence

Future work may integrate reliable temperature, frequency, battery, thermal and other exposed hardware signals.

19.5 Edge AI

The local architecture can provide a path toward compact edge models for anomaly classification, resource prediction, signal processing and fault reasoning.

19.6 Explainability

An evidence graph could explicitly connect an assessment to measurements, derived features, events and contextual changes.

20. Discussion and Conclusion

SystemPulse occupies an intermediate position between conventional monitoring and autonomous system diagnosis. Its current value is the integration of real local telemetry, short-term behavioral comparison, deterministic interpretation, longitudinal research collection and evidence-oriented explanation.

The project demonstrates why system intelligence is fundamentally contextual. A single measurement is often insufficient; recent behavior, workload, process identity, network state, device state and temporal persistence can change the meaning of an observation.

The most important boundary is equally clear: the current system is not yet a complete adaptive anomaly-detection or predictive-diagnostics platform. Long-term drift modeling, multivariate correlation, statistical validation, machine-learning comparison, richer hardware telemetry and predictive analysis remain research directions.

This separation between implemented capability and proposed research is intentional. It provides a credible basis for future experimentation without overstating results that have not yet been measured.

20.1 Research Statement

SystemPulse is an evolving technical research and engineering platform for local system intelligence, behavioral analysis, evidence-oriented reasoning and future edge-oriented anomaly research.

SystemPulse — Local-First System Intelligence and Behavioral Analysis

Author: Nachiket Narkhede

Selected technical references

[1] SystemPulse project source code and README, provided project archive.
[2] SystemPulse technical article, provided project documentation.
[3] Berrada et al., arXiv:1906.06940, system-level provenance anomaly detection.
[4] Krishnamurthy et al., arXiv:2406.12438, dynamic behavioral anomaly analysis.
[5] Rahman, Bandara, Shetty, arXiv:2607.07075, behavioral anomaly detection in network telemetry.
[6] NIST, Artificial Intelligence Risk Management Framework, general trustworthy-AI context.